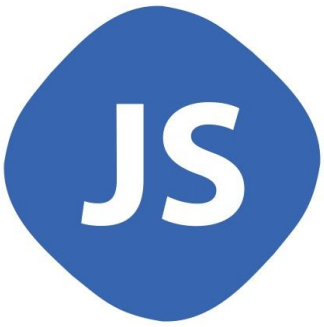


Вложенные циклы

- Введение в вложенные циклы
- Обработка коллекций вложенными циклами
- Обработка сложных структур вложенными циклами
- Обработка файлов текстовых форматов
- Управление циклом. Метка Label

Введение в вложенные циклы

Циклы можно вкладывать один внутри другого, при этом первый цикл называется **внешним** циклом, второй цикл называется **вложенным** (внутренним).

> ***Важно***. При каждой итерации **внешнего** цикла, **вложенный** цикл выполняется **полностью**.

Если **усвоили простые** циклы, здесь тоже все получится. Нет ничего сложного, просто быть внимательным при написании кода и иметь желание его писать. У меня есть, поэтому полетели...

Вложенные циклы можно создавать любой циклической конструкцией.

example_1. Вложенный цикл for

```
<script>

    // внешний цикл for

    for (var i = 1; i < 4; i++) {

        // вывод переменной внешнего цикла - i

        document.write(i + ". Строка ВНЕШНЕГО цикла<p>");

        // внутренний цикл for
```

```
        for (var k = 1; k < 5; k++) {
            // вывод переменной внутреннего цикла - k
            document.write("    " + k + " -
            Строка ВЛОЖЕННОГО цикла<p>");
        }
    }
</script>
```

Переменные внешнего цикла доступны внутри вложенного цикла.

example_2. Переменные внешнего цикла доступны вложенному

```
<script>
    // внешний цикл for
    for (var i = 1; i < 4; i++) {
        // i - переменная внешнего цикла
        // внутренний цикл for
        for (var k = 1; k <= 10; k++) {
            // k - переменная внутреннего цикла
            // выводим произведение переменных i * k
            document.write(i + " * " + k + " = " + i * k, "<br/>"
            );
        }
        // вывод по завершении внутреннего цикла
        document.write("<hr><p>");
    }
</script>
```



Как было сказано, при организации вложенных циклов позволительна любая комбинация циклических конструкций. В примере **example_3** для организации вывода используем две конструкции **while**.

example_3. Во вложенных структурах доступен любой набор циклов

```
<script>

    // инициируем переменную внешнего цикла

    let i = 1;

    while (i < 5) {

        // инициируем переменную внутреннего цикла

        let k = 1;

        while (k <= 10) {

            // выводим произведение переменных i * k

            document.write(i + " * " + k + " = " + i * k, "<br/>"
            );

            k++; // инкремент внутренней переменной

        }

        // вывод по завершении внутреннего цикла

        document.write("<hr><p>");

        i++; // инкремент внешней переменной

    }

</script>
```

В примере **example_4** внешний цикл использует конструкцию **while**, вложенный - конструкцию **for**.

example_4. Переменные циклов

```
<script>

    // инициируем переменную внешнего цикла
```

```
    let i = 1;
    while (i < 5) {
        for (k=1; k <= 10; k++) {
            // выводим произведение переменных i * k
            document.write(i + " * " + k + " = " + i * k, "<br/>"
            );
        }
        // вывод по завершении внутреннего цикла
        document.write("<hr><p>");
        i++; // инкремент внешней переменной цикла
    }
</script>
```

Обработка коллекций вложенными циклами

Примеры вложенных циклов — это замечательно. Осталось определить, каким образом эти знания можно применить на практике. Рассмотрим обработку коллекций с помощью вложенных циклов.

Обработка элементов коллекций зависит от **предметной** области, поставленной **задачи**, **HTML-разметки** и много от чего еще. Но я попробую придумать себе задачу и успешно ее решить (если не получится, я поменяю задачу, но фразу про ее решение, пожалуй, оставлю ;)

Итак, мы имеем HTML-код в виде списка рейтинга продаж смартфонов (**example_5**). В этом коде, как ни крути, вложенному циклу места нет, слишком все просто и очевидно. Создаем коллекцию, разбираем, выводим.

```
<div id="top">
```

```
        <ol>
            <li>Xiaomi 11T Pro</li>
            <li>OnePlus 9RT</li>
            <li>Samsung Galaxy S21 FE</li>
            <!-- ... -->
        </ol>
</div>
```

example_5. Простые коллекции выводятся просто)

```
<script>
    // получение коллекции элементов
    var elements =
    document.getElementById('top').firstElementChild.children;
    // количество элементов коллекции
    let len = elements.length;
    // вывод в цикле for
    for (var el of elements) {
        document.write (el.innerText, "<br/>");
    }
</script>
```

Сохранение таблицы в двумерный массив

Табличная разметка из кода демонстрационного примера **example_6** выглядит уже достаточно сложно. Таблица состоит из **набора строк**, строка из **набора ячеек**. Вот теперь вложенному массиву самое место.

Примечание. Примеры, рассматриваемые далее уже достаточно сложные и требуют знаний следующих тем и уроков:

- **DOM** – Document Object Model

- **Простые** циклы.
- **Специальные** циклы.

Ниже представлен фрагмент HTML-разметки рассматриваемой таблицы.

```
<table>
    <tr>
        <td>1</td>
        <td>Atom Heart Mother</td>
        <td>10 октября 1970</td>
        <td>EMI, Harvest, Capitol</td>
        <td>LP, CD</td>
        <td>Золотой (USA</td>
    </tr>
...
</table>
```

Внешний цикл реализован конструкцией **while** (не обязательно), внутренний циклом - **for** (не обязательно). Выполните анализ алгоритма вывода. Наберите код примера, разберитесь с каждой строкой кода. Используйте консоль для вывода интересующих вас переменных и коллекций.

При наборе кода будьте внимательны, следите за правильным написанием используемых конструкций.

example_6. Сохранение таблицы в массив

```
<script>
    // получение коллекции строк таблицы
    let elements = document.querySelectorAll('tr');
    // количество элементов коллекции
    let len = elements.length;
    // инициируем переменные
```

```
    let i = 0; // итератор цикла
    let arr = []; // одномерный массив для хранения табличных
    данных
    // цикл while
    while (i < len) {
        // получаем коллекцию элементов td для каждой строки tr
        let td = elements.item(i).children;
        // значением i-го элемента массива будет - массив
        arr[i] = [];
        // цикл for
        for (j=0; j<td.length; j++) {
            arr[i][j] = td.item(j).innerHTML;
        }
        // переход к следующей строке таблицы
        i++; // приращение итератора цикла
    }
    console.log(arr);
</script>
```

Сохранение вложенного списка в массив

И еще немного усложним задачу. В демонстрационном примере **example_7** обработаем и сохраним в массив данные вложенного списка.

> ***Примечание***. Будьте внимательны. При объяснении примера будут задействованы термины из **теории баз данных**.
>
> Дело в том, что асинхронное взаимодействие предполагает получение данных из **серверного сценария**. Информация на сервере хранится в

базах данных, следовательно логическую структуру принимаемых клиентом данных будет определять модель используемой СУБД.

Ниже представлен фрагмент HTML-разметки рассматриваемого вложенного списка.

```
<ol>
      <li>Гарри Поттер и узник Азкабана
            <ul>
                  <li>Джоан Роулинг</li>
                  <li>978-389-17021-6</li>
                  <li>2019</li>
                  <li>Махаон</li>
            </ul>
      </li>
      <!-- ... -->
      <li>Прислуга
            <ul>
                  <li>Кэтрин Стокет</li>
                  <li>978-5-86471-732-5</li>
                  <li>2016</li>
                  <li>Фантом Пресс</li>
            </ul>
      </li>
</ol>
```

За элементами списка **li первого** уровня следует **вложенный список** элементов списка **li второго** уровня (**ul**). Это значит, что для обработки коллекции может понадобиться вложенный цикл.

Для наглядности разобью пример на несколько частей, выполним **пошаговый анализ** обработки элементов коллекции на каждом этапе кодирования.

example_7-1. Запись в массив названий книг

```
<script>
    // массив хранения списка
    var arr = [];
    // получение HTML-коллекции элементов li
    let elements = document.querySelectorAll('ol > li');
    // убедитесь, понимаете ли вы, какая коллекция отобрана
    // console.log(elements);
    // цикл for по элементам коллекции
    for (var i = 0; i < elements.length; i++) {
        // текстовый узел - название фильма
        console.log (elements[i].firstChild.data);
        arr[i] = elements[i].firstChild.data;
    };
    console.log("Массив хранения списка");
    console.log(arr);
</script>
```

На каждой итерации по элементам исходной коллекции выбираем элементы **li вложенного списка**. Выполняем цикл по элементам вложенного списка.

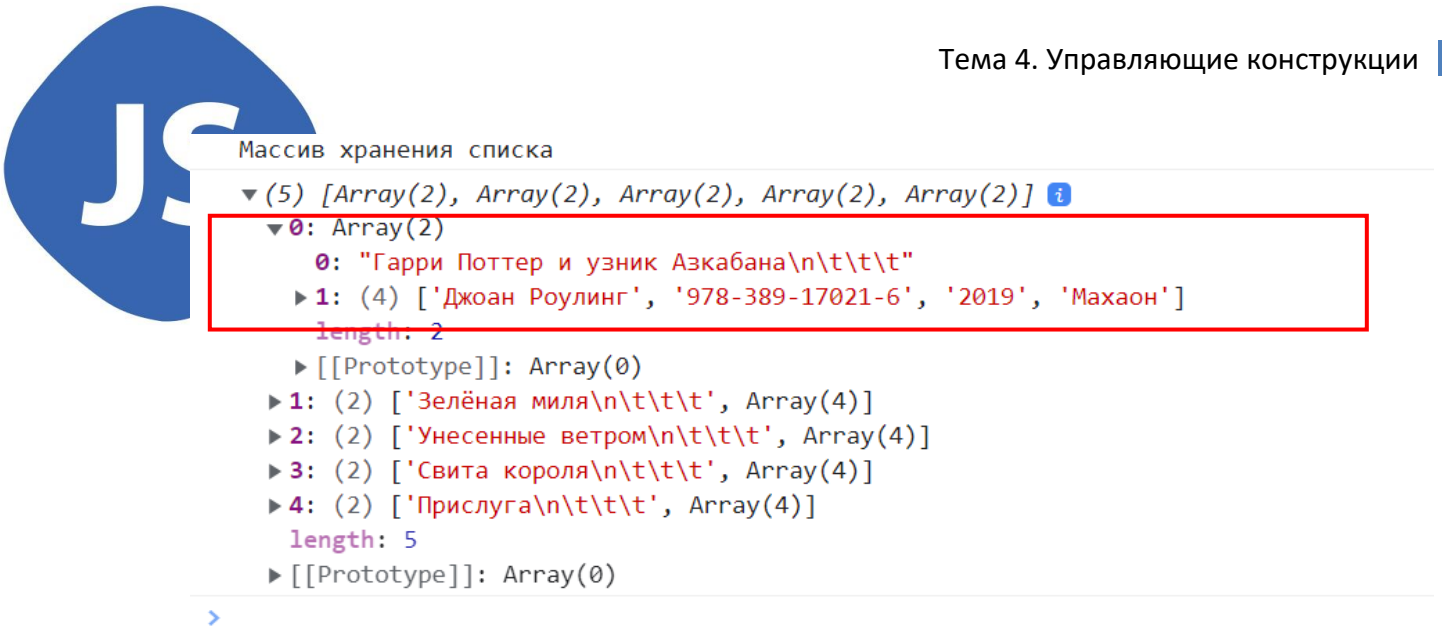
example_7-2. Получение данных вложенного списка

```
<script>
    // массив хранения списка
    var arr = [];
    // получение HTML-коллекции элементов li
    let elements = document.querySelectorAll('ol > li');
```

```
    // цикл for по элементам коллекции
    for (var i = 0; i < elements.length; i++) {
        arr[i] = elements[i].firstChild.data;
        // выбираем ul > li вложенного списка на каждой итерации
        list = elements[i].firstElementChild.children;
        // цикл по элементам li вложенного списка
        for(var k = 0; k < list.length; k++){
            // выводим каждый вложенный элемент li
            console.log(list[k].firstChild.data);
        }
        // разделитель между вложенными элементами ul
        console.log("----------");
    };
    console.log("Массив хранения списка");
    console.log(arr);
</script>
```

Элементы вложенного списка собираем в отдельный массив (массив второго уровня, подмассив). Таким образом основной массив состоит из **двух элементов**:

- 0 – **Название книги** (скалярное значение)
- 1 – **Элементы вложенного списка** (массив)



Будьте внимательны при написании кода. Обратите внимание, на то, каким образом формируется основной массив и на **обнуление вложенного подмассива** на каждой итерации по коллекции внешнего списка.

example_7-3. Сохранение вложенного списка в массив

```
<script>

    // массив хранения списка

    var arr = [];

    // получение HTML-коллекции элементов li внешнего списка

    let elements = document.querySelectorAll('ol > li');

    // цикл for по элементам коллекции

    for (i = 0; i < elements.length; i++) {

        // массив хранения вложенного списка объявлен внутри
        цикла,

        // потому, что ->

        // для каждого вложенного списка должен быть свой массив
        subarr

        var subarr = [];

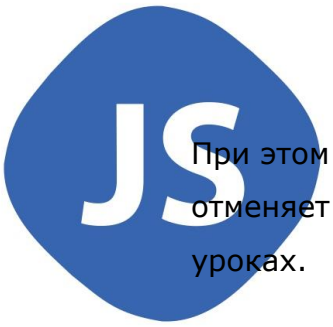
        arr[i] = [];
```

```
        // первый элемент подмассива - скалярное значение
        // -> Название книги
        arr[i][0] = elements[i].firstChild.data;
        // выбираем ul > li вложенного списка на каждой итерации
        list = elements[i].firstElementChild.children;
        // цикл по элементам li вложенного списка
        for(k = 0; k < list.length; k++){
            // собираем в массив каждый вложенный элемент li
            subarr[k] = list[k].firstChild.data;
        }
        // второй элемент подмассива - тоже массив
        // -> Элементы вложенного списка
        arr[i][1] = subarr;
    };
    console.log("Массив хранения списка");
    console.log(arr);
</script>
```

Примечание. В названии книг присутствуют **управляющие символы переноса строки** и **табуляции** ("\n\t\t\t"). Происхождение символов вызвано имеющейся верстской списка (тема: DOM – Document Object Model).

В одной из самостоятельных работ рассмотрим один из способов избавления от нежелательных элементов массива данных.

Таким образом выполняется обработка вложенных данных, не обязательно коллекций, это могут быть любые **структурированные данные**. Какова бы ни была сложность структуры, алгоритм обработки от этого не меняется.



При этом **цикл** обхода коллекции может быть **любым**, что, впрочем, не отменяет вопроса **оптимальности**, но этим будем заниматься на следующих уроках.

Все это хорошо, но посмотрим на обрабатываемый циклом фрагмент HTML-разметки еще раз.

Вот характеристика вложенного списка:

- количество пунктов списка везде одинаково,
- последовательность перечисления одинаково,
- данные каждого пункта списка - простая строка,
- данные атомарны и неделимы (делимость имени автора не учитываем).

Структура списка – **константа**, посмотрим, можно ли обойтись **без усложнения** кода дополнительными циклическими конструкциями, и сделать так, как предложено в следующем примере.

example_8. Иногда вложенная структура необоснованно усложняет код

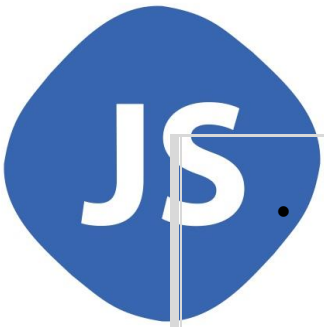
```
<script>
    // массив хранения списка
    var arr = [];
    // получение HTML-коллекции элементов li внешнего списка
    let elements = document.querySelectorAll('ol > li');
    // цикл for по элементам коллекции
    for (i = 0; i < elements.length; i++) {
        // массив хранения вложенного списка объявлен внутри
        цикла,
        // потому, что ->
        // для каждого вложенного списка должен быть свой массив
        subarr
        var subarr = [];
        arr[i] = [];
```

```
        // первый элемент подмассива - скалярное значение
        // -> Название книги
        arr[i][0] = elements[i].firstChild.data;
        // выбираем ul > li вложенного списка на каждой итерации
        list = elements[i].firstElementChild.children;
        // избавились от вложенного цикла
        // иногда так легче читается код и больше маневра для
        верстки
        subarr[0] = list[0].firstChild.data;
        subarr[1] = list[1].firstChild.data;
        subarr[2] = list[2].firstChild.data;
        subarr[3] = list[3].firstChild.data;
        // второй элемент подмассива - тоже массив
        // -> Элементы вложенного списка
        arr[i][1] = subarr;
    };
    console.log("Массив хранения списка");
    console.log(arr);
</script>
```

Все очень хорошо и наглядно получается. Рассуждать о том, как делать правильно не имеет смысла. Правильно делать так, чтобы код хорошо читался, был оптимальным и выполнялся без сбоев.

Важно. Общий принцип разработчика прост.

- **По объектам массива** (товары, заказы, книги...) **перемещаемся с помощью циклов**.
- **Информацию по конкретному объекту** (свойства объекта)



- **выводим вручную**. Например, с помощью HTML-карточки.
- Если одно или несколько свойств объекта представлены в виде **массива** – **снова используем цикл**.

А что делать фронтендеру, если данные с сервера пришли, например, в таком виде:

```
<ol>
      <li>Гарри Поттер и узник Азкабана
            <ul>
                  <li>Джоан Роулинг</li>
                  <li>978-389-17021-6</li>
                  <li>2019</li>
                  <li>Махаон</li>
            </ul>
      </li>
      <!-- ... -->
      <li>Прислуга
            <ul>
                  <li>Фантом Пресс</li>
                  <li>Кэтрин Стокет</li>
                  <li>978-5-86471-732-5</li>
            </ul>
      </li>
</ol>
```

Ваш код с таким списком работать не будет.

К сведению. А делать ничего не надо. Точнее так, делать нужно минимум – настучать начальнику на бэкендера, и тот его накажет... ;)

Примечание. Все примеры выполняются не для того, чтобы взять табличное представление или список коллекции и вывести в виде двумерного массива.

Все делается для того, чтобы научиться **получать**, **хранить**, **обрабатывать** и **выводить данные**, **представленные либо в виде HTML-коллекций, либо в виде JS-массивов или объектов**.

Когда начнутся уроки взаимодействия с сервером, все встанет на свои места.

Мы рассмотрели общие принципы управления коллекциями с помощью циклов. Закрепим материал в самостоятельных заданиях.

Обработка сложных структур вложенными циклами

В следующем примере выведем два первых элемента вложенного массива в виде списка первого уровня, остальные элементы, сколько бы их не было – в виде списка второго уровня.

Вывод двумерного массива в список

example_9. Вывод двумерного массива в список

```
<script>
    let data = document.getElementById("data");
    let list = "";
    for (let i = 0; i < arr.length; i++) {
        // в список первого уровня забрали
        // arr[i][0] - идентификатор альбома
        // arr[i][1] - название альбома
```

```
            list += `<li>${arr[i][1]} (id = ${arr[i][0]})</li>`;
            // инициируем переменные для вложенного цикла
            // так как индексы 0 и 1 ушли в список первого уровня
            // начинаем счет с индекса - 2
            let j = 2;
            let subList = "";
            // в этот раз используем цикл do (совсем не обязательно)
            do {
                subList += `<li>${arr[i][j]}</li>`;
                j++;
            }
            while (j < arr[i].length);
            list += `<ul>${subList}</ul>`;
        }
        data.insertAdjacentHTML('afterbegin', `<ol>${list}</ol>`);
</script>
```

Представленный код вовсе не отменяет доктрину об одинаковом количестве пунктов (свойств) представления объекта (количество элементов первого подмассива). Это исключение, подчеркивающее важность правила хранения данных объектов.

Вывод массива объектов в таблицу

Выведем массив объектов (форма наиболее частого представления данных), в таблицу документа.

При динамическом обходе свойств объектов удобно использовать альтернативный доступ к свойствам объекта через **квадратные скобки**.

Важно. **Квадратные скобки** позволяют обратиться к свойству, имя которого может быть **результатом выражения**. Например, имя свойства может храниться в **переменной.**

Тестируем динамическое формирование ключей объекта.

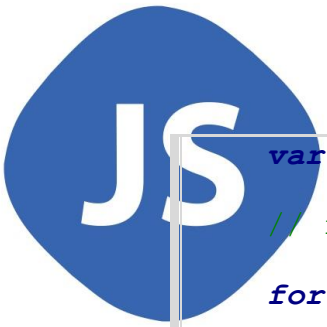
example_10. Доступ к свойствам объекта через квадратные скобки

```
<script>
    let user = {
        name: "John",
        login: "master",
        age: 30
    };
    //
    console.log(user.name); // John
    console.log(user.login); // master
    console.log("----------");
    // динамическое формирование ключа
    let key = "name";
    console.log(user.key); // undefined
    console.log(user[key]); // John
</script>
```

В большинстве случаев, когда имена свойств известны используется запись через **точку**. Если же нам нужно что-то более сложное, то мы используем **квадратные скобки**.

example_11. Динамический вывод массива объектов

```
<script>
```



```
var tbody = "";
// перебор объектов массива courses
for (obj of courses) {
    let str = "";
    // перебор ключей объекта
    for (key in obj) {
        str += `<td>${obj[key]}</td>`;
    }
    // console.log(obj);
    // формируем строки таблицы
    tbody += `<tr>${str}</tr>`;
}
// формируем контент страницы
let content = `
    <table border=1>
    <tr>
    <th>ID</th>
    <th>Название</th>
    <th>Время обучения</th>
    <th>Расписание</th>
    <th>Уровень</th>
    <th>Преподаватель</th>
    <th>Цена</th>
    </tr>
    ${tbody}
    </table>`;
```



```
    document.write(`${content}`);
</script>
```

Обработка файлов текстовых форматов

При работе с приложениями в асинхронных режимах вы будете **передавать**, **получать**, **обрабатывать**, **хранить** наборы данных. Данные циркулируют между приложениями в текстовых форматах. Сложного ничего нет, но урок не будет полным, если я не предложу рассмотреть пару примеров.

Файлы CSV

CSV (от англ. Comma-Separated Values — значения, разделённые запятыми) — **текстовый формат**, предназначенный для **представления табличных данных**. Строка таблицы соответствует строке текста, которая содержит одно или несколько полей, разделенных запятыми.

Формат CSV стандартизирован не полностью. Идея использовать запятые для разделения полей очевидна, но при таком подходе возникают проблемы, если исходные табличные данные содержат запятые или переводы строк.

Многие приложения, которые работают с форматом CSV, **позволяют выбирать символ разделителя и символ кавычек**.

example_12. Сохранение CSV файла в массив

```
<script>
    // разбиваем исходную строку построчно в массив
    // разделителем является символ переноса строк '\n'
    let groups = data.split("\n");
    // console.dir(groups);
    let arr = [];
```

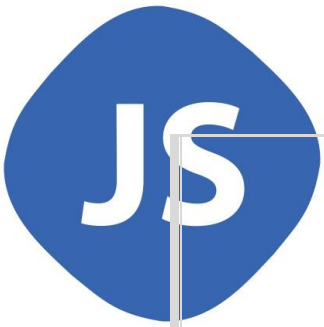
```
    i = 0;
    for (line of groups) {
        // разбиваем каждый элемент массива groups в массив
        // разделителем является символ ';'
        arr[i] = line.split(";");
        i++;
    }
    console.log(arr);
</script>
```

Файлы JSON

Мы уже много работали с JSON форматом, теперь попробуем **автоматизировать** процесс обработки его структуры.

example_13. Вывод JSON файла в браузер

```
<script>
    let data = document.getElementById("data");
    teams = JSON.parse(teams);
    // console.log(teams);
    let out = "";
    for (let i = 0; i < teams.length; i++) {
        out = `
            <h3>${teams[i].name} (id = ${teams[i].id})</h3>
            Алиас: ${teams[i].alias}<br>
            Страна: ${teams[i].country}<br>
            Описание: ${teams[i].content}<br>
            Дата основания: ${teams[i].date}<br>
```



```
            <b>Альбомы:</b>
        `;
        var albums = teams[i].albums;
        let list = '';
        for (album of albums) {
            list += `
            <li>
            ${album.name} (Количество треков: ${album.track},
            Дата написания: ${album.date})
            </li>
            `;
        }
        out += '<ol>' + list + '</ol><hr>';
        data.insertAdjacentHTML('beforeend', out);
    }
</script>
```

JSON может быть достаточно сложной структурой, в таком случае, без вложенных массивов скорее всего, вам не обойтись.

Управление циклом

В прошлом уроке коснулись такого понятия, как **метка**. Метка представляет собой оператор с идентификатором, который позволяет ссылаться на какое-то место в программе.

Вы можете использовать метку, чтобы обозначить цикл, а затем использовать операторы **break** или **continue**, чтобы указать, должна ли программа прерывать цикл или продолжать его выполнение.



Синтаксис

```
myLabel: {
      statement;
}
```

Именем метки может быть любой **корректный** JavaScript **идентификатор**, не являющийся зарезервированным словом. Оператор, указанный вами после метки, может быть **любым выражением**.

Пример использования метки:

```
myLabel: while (condition) {
      if (condition 2) {
            break myLabel;
      }
}
```

Вызов **break myLabel** приведёт к выходу из цикла, перед которым указана метка **myLabel**.

Если метка используется с ключевым словом **continue**, то в этом случае выполнение этого действия приведёт к немедленному переходу к **следующей итерации цикла**, перед которым данная метка указана.

> ***Примечание***. В коде с простым циклом использование метки не даст никакого результата. Её есть смысл использовать только при работе с **вложенными циклами**.

В примере **example_14** вызов оператора **break** и оператора с меткой **break inner** приведет к одному и тому же эффекту – выход из *ближайшего* (внутреннего) цикла.

example_14. Прерывание выполнения внутреннего цикла

```
<script>
```

```
    // метка внешнего цикла
    outer:
    for(var i = 0; i <= 3; i++) {
        // метка внутреннего цикла
        inner:
        for(var j = 0; j <= 3; j++) {
            // каждый раз при j > 3 внутренний цикл будет
            прерываться
            // if (j > 3) break; // выход из внутреннего цикла
            // аналогично, но с использованием метки inner:
            if (j > 3) break inner // выход из внутреннего цикла
            document.write("i = " + i + "; j = " + j + "<br>");
        }
    } // end for
    // первая инструкция после цикла for
    // всегда выполнится
    document.write("<hr>FINAL i = " + i + "; j = " + j + "<br>");
</script>
```

Вот теперь вызов оператора с меткой приведет не просто к выходу из цикла, а к выходу из **указанного меткой цикла**.

example_15. Прерывание выполнения внешнего цикла

```
<script>
    // метка внешнего цикла
    outer:
    for(var i = 0; i <= 3; i++) {
        // метка внутреннего цикла
```

```
        inner:
        for(var j = 0; j <= 3; j++) {
            if (i == 2) break outer; // выход из внешнего цикла
            document.write("i = " + i + "; j = " + j + "<br>");
        }
    } // end outer:for
    // первая инструкция после цикла outer:for
    // всегда выполнится
    document.write("<hr>FINAL i = " + i + "; j = " + j + "<br>");
</script>
```

И в последнем примере метка **outer** обозначает конструкцию – блок **{}**. А это значит, что оператор с меткой **break outer** приведет к выходу из всего блока.

example_16. Прерывание выполнения блока кода

```
<script>
    // метка внешнего цикла
    // поместим инструкции внешней метки в блок {} (!)
    outer: {
    for(var i = 0; i <= 3; i++) {
        // метка внутреннего цикла
        inner:
        for(var j = 0; j <= 3; j++) {
            if (i == 2) break outer; // выход из внешней метки
            outer:
            document.write("i = " + i + "; j = " + j + "<br>");
        }
    } // end outer:for
```

```
    // теперь эта инструкция не выполнится
    document.write("<hr>FINAL i = " + i + "; j = " + j + "<br>");
    } // конец блока кода
</script>
```

Операторы **break** и **continue нельзя** использовать в выражениях тернарных операторов.

В **самостоятельных работах** к уроку рассмотрим следующие вопросы:

- Оптимизация структуры DOM документа.
- Обработка вложенными циклами файлов формата CSV.
- Обработка вложенными циклами массива объектов.
- Обработка вложенными циклами массива массивов.